Noor ul Huda

Section 5
Task 1

GETTING STARTED ON EDAPLAYGROUND

18-07-2026

PLAYGROUND LINK: https://edaplayground.com/x/S7Px

GIT LINK: https://github.com/Nooorrr66/MEDS_2026/tree/main/module_4/5_edaplayground

CODE:

Design:

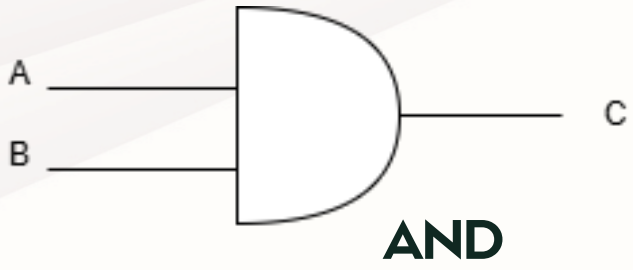
```
module and_gate(
    input logic a,
    input logic b,
    output logic y
);

assign y = a & b;

endmodule
```

Test Bench

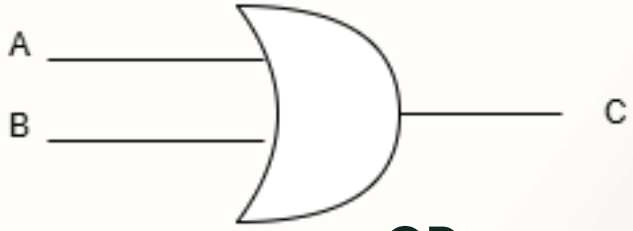
```
module tb;

logic a;
logic b;

logic y;

// Extra signals
logic or_out;
logic xor_out;
```

```
and_gate dut(
 .a(a),
 .b(b),
 .y(y)
);

assign or_out = a | b;
assign xor_out = a ^ b;

initial begin

 $dumpfile("dump.vcd");
 $dumpvars(0,tb);

 a=0; b=0;
 #10;
 $display("a=%0b b=%0b AND=%0b
OR=%0b XOR=%0b",
 a,b,y,or_out,xor_out);

 a=0; b=1;
 #10;
 $display("a=%0b b=%0b AND=%0b
OR=%0b XOR=%0b",
 a,b,y,or_out,xor_out);

 a=1; b=0;
 #10;
 $display("a=%0b b=%0b AND=%0b
OR=%0b XOR=%0b",
 a,b,y,or_out,xor_out);

 a=1; b=1;
 #10;
 $display("a=%0b b=%0b AND=%0b
OR=%0b XOR=%0b",
 a,b,y,or_out,xor_out);

 $finish;

end

endmodule
```

SCHEMATIC:



AND

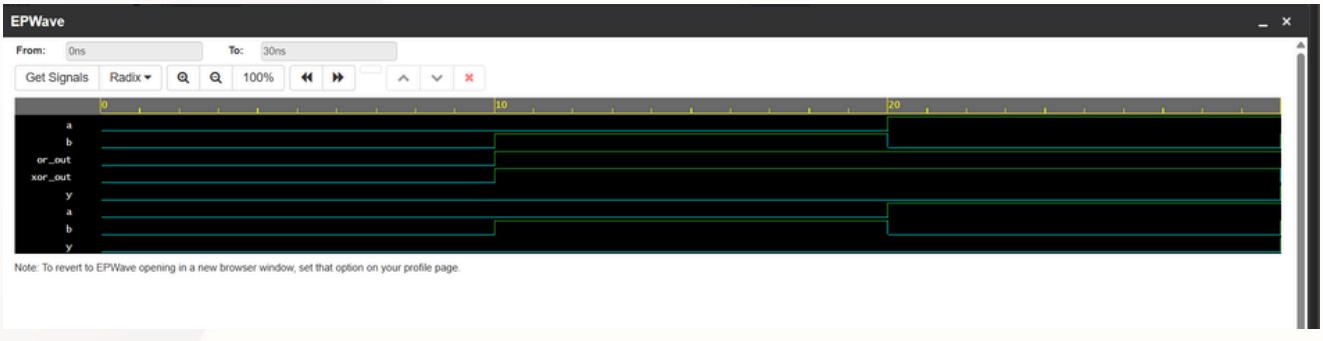


OR

XOR

TERMINAL:

WAVEFORM:



EXPLANATION:

The design implements a two-input AND gate using a continuous assignment. The testbench applies all four input combinations and additionally computes the OR and XOR outputs for comparison. The waveform confirms that the outputs follow the expected truth tables for all combinations.

The module and_gate implements a simple two-input AND gate using a continuous assignment.

- input logic a, b declares the two input signals.
- output logic y declares the output signal.
- assign y = a & b; continuously assigns the logical AND of a and b to the output y. Whenever either input changes, the output updates automatically without requiring a clock.

The testbench verifies the functionality of the AND gate by applying all four possible combinations of the two inputs.
It also computes the OR and XOR operations using continuous assignments:

```
assign or_out  = a | b;
assign xor_out = a ^ b;
```

The simulation proceeds as follows:

- Initializes waveform dumping using $dumpfile and $dumpvars.
- Applies each input combination (00, 01, 10, 11).
- Waits 10 ns after each change.
- Prints the AND, OR, and XOR outputs using $display.
- Ends the simulation with $finish.

This ensures every possible input combination is tested.

The waveform verifies that the circuit behaves according to the truth table over time.

- At 0 ns, both inputs are 0, so the outputs are AND = 0, OR = 0, and XOR = 0.
- At 10 ns, the inputs become 0 and 1. The AND output remains 0, while OR and XOR become 1.
- At 20 ns, the inputs are 1 and 0. The outputs remain the same as the previous case because the logic operation is symmetric.
- At 30 ns, both inputs are 1. The AND output changes to 1, the OR output remains 1, and the XOR output returns to 0 because both inputs are equal.

The waveform confirms that each output transitions exactly as expected for every input combination.

Task 2

17-07-2026

PLAYGROUND LINK: https://edaplayground.com/x/e3bt

CODE:

Design:

“Same as Task 1”

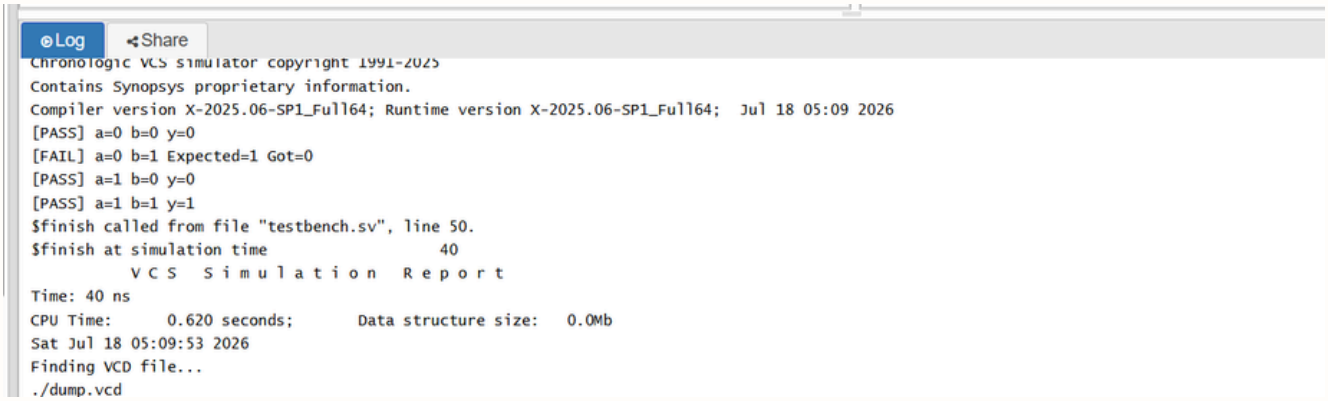
Test Bench

```
module tb;

logic a, b;
logic y;

integer i;

and_gate dut (
    .a(a),
    .b(b),
    .y(y)
);

// PASS/FAIL checker
task check;
    input logic expected;

    begin
        if (y === expected)
            $display("[PASS] a=%0b b=%0b y=%0b", a,
b, y);
        else
            $display("[FAIL] a=%0b b=%0b
Expected=%0b Got=%0b",
                     a, b, expected, y);
    end
endtask

initial begin

    $dumpfile("dump.vcd");
    $dumpvars(0, tb);

    // Test all four input combinations
    for (i = 0; i < 4; i = i + 1) begin

    a = i[1]; // MSB
    b = i[0]; // LSB

    #10;

    //check(a & b); if we add this we get the correct
testbench, the below one is for getting one fail
    if (a == 0 && b == 1)
    check(1); // Wrong on purpose
    else
    check(a & b);

    end

    $finish;

end

endmodule
```

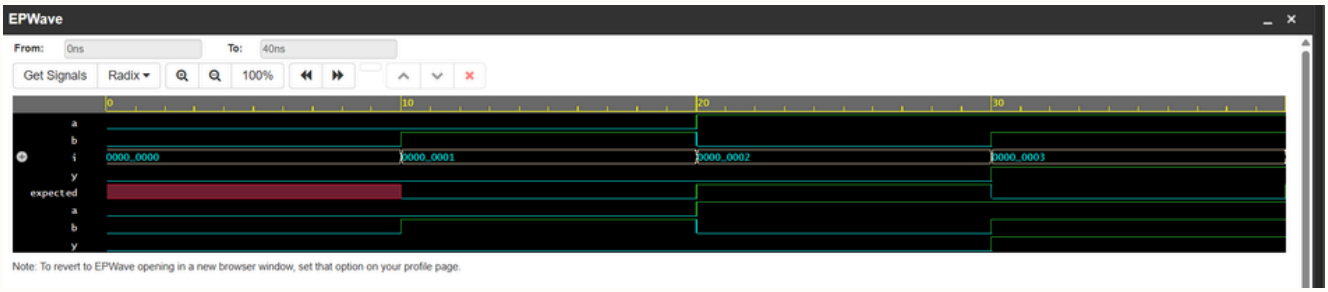
SCHEMATIC:

Same as task 1

TERMINAL:



WAVEFORM:



EXPLANATION:

In this task, the original testbench was converted into a self-checking testbench by introducing a check task that automatically verifies the output of the design. A for loop was used to exhaustively test all possible input combinations, reducing manual effort and improving code readability. The simulation results showed that the checker successfully detected incorrect outputs when an intentional error was introduced and reported [PASS] for all test cases after the error was corrected. This demonstrates the advantage of self-checking testbenches in automating functional verification.